Graded Project, Part 3

Explainable AI with SHAP

Why does the model think a RetailGenius customer is about to leave?

Group: Group 5

Members: Aravind Reddy Kolan, Prem Siva Naga Sai Ram Kumar Gudivada, Praveen Kumar Koppula

Date: November 2026

GitHub repository: https://github.com/premgudivada7/ai-project-methodolagy- (same repository as Part 2)

1. Why we need explanations

The churn model from Part 2 is a random forest with hundreds of trees. It predicts well, but nobody can read 500 trees to understand a single score. That is a real problem for RetailGenius. A marketing manager will not spend budget on a customer just because "the model said 0.87". A support agent needs to know whether to talk about delivery, a complaint or a discount. And under GDPR, customers can ask how automated decisions about them were made.

SHAP (SHapley Additive exPlanations) answers this. It comes from game theory: each feature is treated as a "player" and gets a share of the prediction, called its Shapley value. For every customer, the SHAP values add up exactly to the difference between the model's prediction for that customer and the average prediction (the base value). A positive value pushes towards churn, a negative one pushes away from it.

2. Setting up SHAP in the project

We added shap to pyproject.toml and requirements.txt, so it installs with the rest of the project. All the XAI code is in churn/explain.py and runs as one step of the pipeline:

```
pip install shap            # already in requirements.txt / pyproject.toml
make explain                # = python -m churn.explain
mlflow run . -e explain     # same thing as an MLflow Project entry point
python scripts/build_xai_report.py   # rebuilds this report from the outputs
```

The script loads the champion model from the MLflow Model Registry (models:/retailgenius-churn-model@champion), applies the pipeline's preprocessing to the test customers so we explain the model on the same features it really sees, and builds a TreeExplainer on the random forest:

```
explainer = shap.TreeExplainer(clf)       # exact, fast algorithm for tree models
explanation = explainer(X_test_transformed)  # shape (customers, features, 2 classes)
exp0, exp1 = explanation[:, :, 0], explanation[:, :, 1]
```

TreeExplainer computes exact Shapley values for tree ensembles in polynomial time, so we can explain every test customer in seconds instead of approximating with sampling. For a random forest the values are in probability units. All figures are saved in reports/figures/shap and logged to MLflow under a run named "shap-explanations".

We explained 1015 test customers on 31 features (after one-hot encoding). The base value for the churn class is 0.500, which is the average churn probability the model gives on the training data. It is close to 0.5 rather than the real churn rate (about 17%) because we trained with balanced class weights, which make the forest treat both classes as equally frequent.

3. Explaining one customer

By default the script picks the real churner with the highest predicted risk in the test set. Here that is customer 55168. The model gives them a churn probability of 0.95, and they did churn. (Another customer can be chosen with python -m churn.explain --index N.)

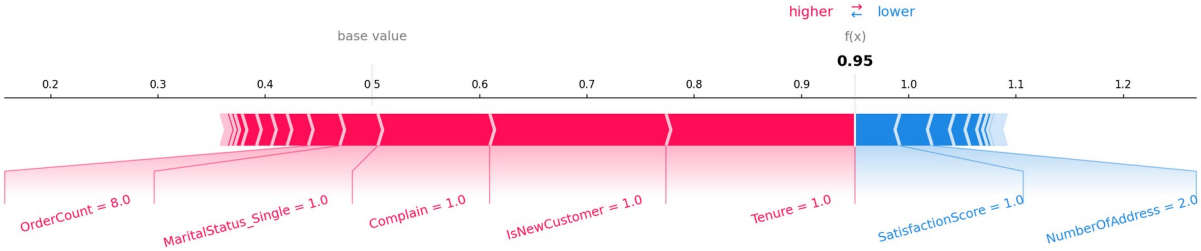


Figure 1 – Force plot for the selected customer (interactive version: 01_force_plot_single_customer.html)

How to read it: the model starts from the base value. Red arrows are features that push the probability up, blue arrows push it down, and the width of each arrow is the size of the effect. Where they meet is the final prediction.

The three biggest reasons this customer is flagged:

- Tenure = 1.00 (+0.176)
- IsNewCustomer = 1.00 (+0.164)
- Complain = 1.00 (+0.104)

Each line shows the customer's value for the feature (after imputation) and, in brackets, how much it adds to the churn probability.

4. Explaining all customers at once

If we take the force plot of every customer, turn it vertical and put them side by side, we get the stacked force plot. It is interactive, so it is saved as HTML (02_force_plot_all_customers.html). Customers can be ordered by similarity, by prediction or by a single feature, which makes groups of similar churners easy to spot.

Since an HTML file cannot go in a report, we also made a heatmap that shows the same information: one column per customer, one row per feature, and the colour is the SHAP value. The line on top is the model's output for each customer.

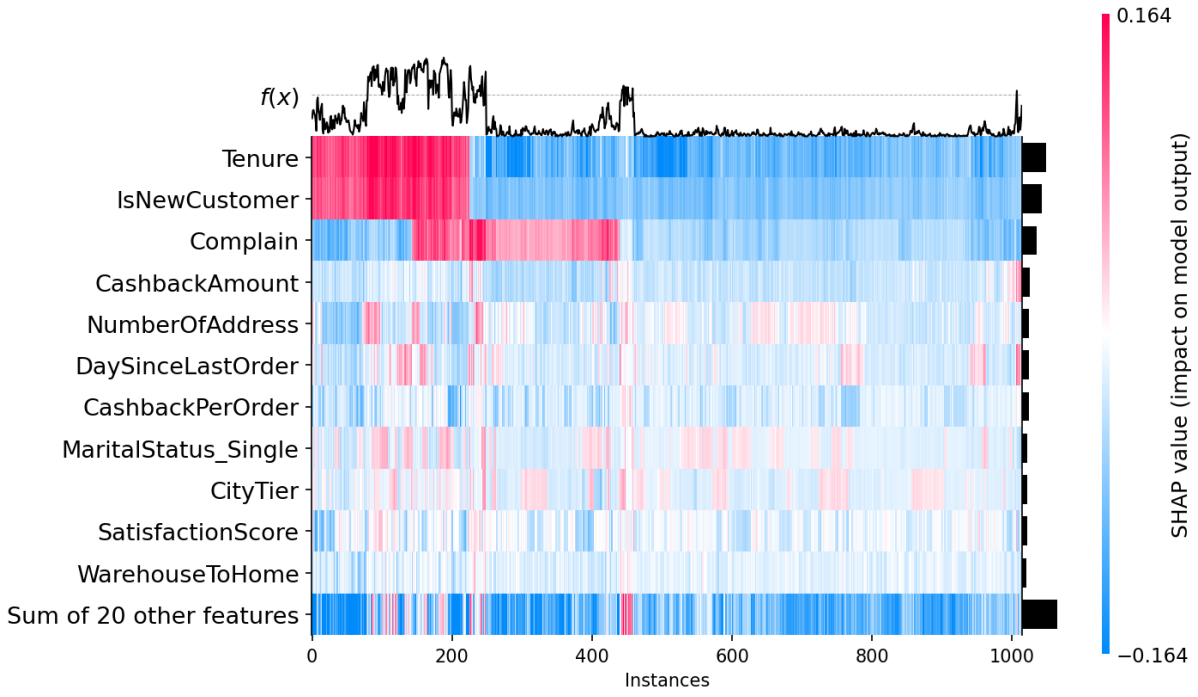


Figure 2 – SHAP heatmap of all test customers

5. Summary plot for each class

The random forest gives one probability per class, so SHAP gives one set of values per class too. In a binary problem the two are mirror images: anything that raises the probability of churning lowers the probability of staying by exactly the same amount. We still plot both, as asked.

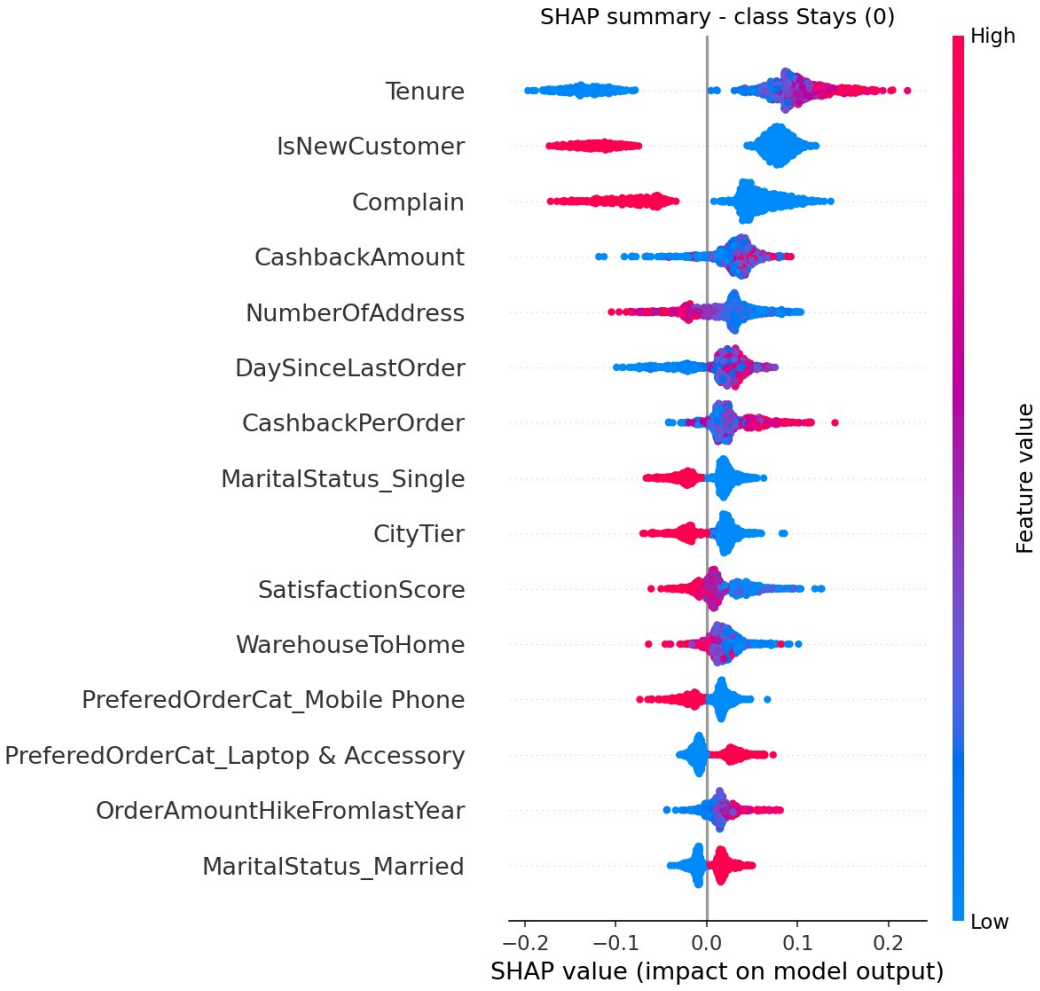


Figure 3 – Summary plot, class 0 (customer stays)

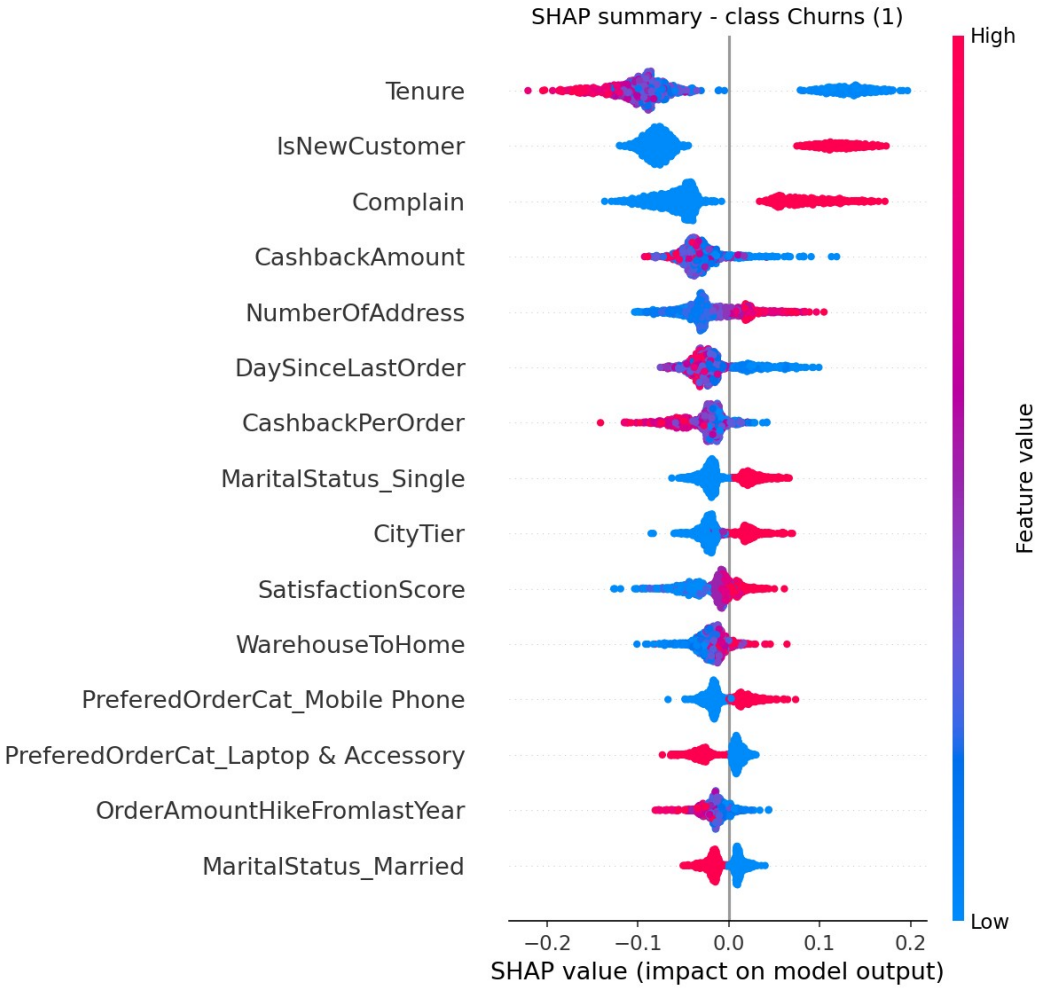


Figure 4 – Summary plot, class 1 (customer churns)

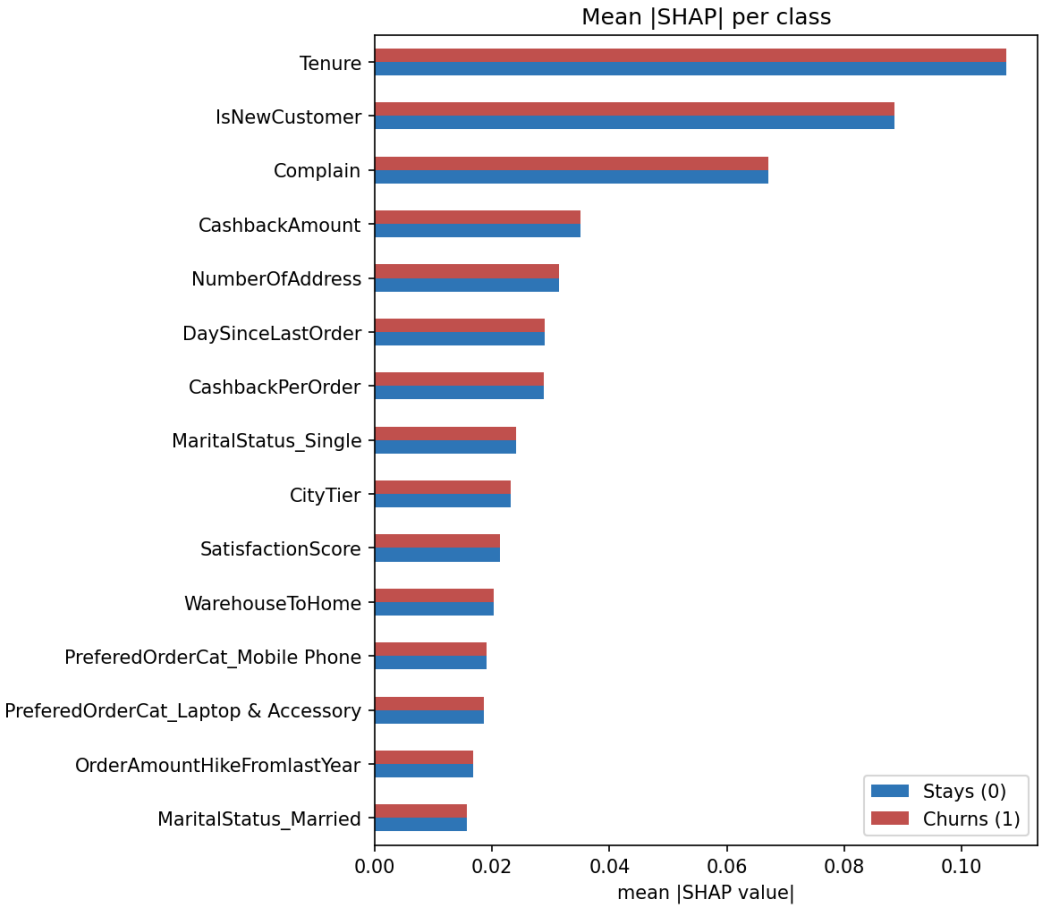


Figure 5 – Mean |SHAP| per feature, both classes

Each dot is one customer. Its position is the SHAP value (its push on the prediction) and its colour is the feature value, red for high and blue for low. Features are sorted from most to least important.

6. Going further for one customer and globally

6.1 Waterfall plot

The waterfall plot is the easiest one to show to a non-technical audience. It starts at the base value at the bottom and adds one feature at a time until it reaches the customer's prediction at the top. Each bar is labelled with the feature value and its contribution.

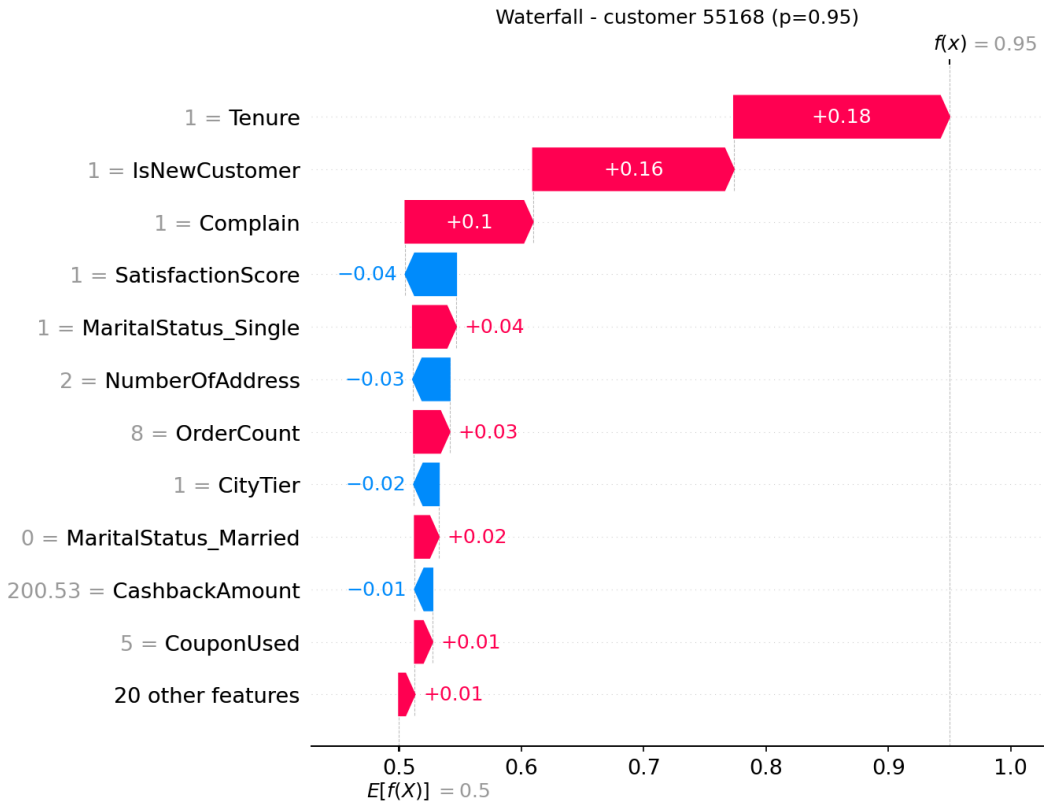


Figure 6 – Waterfall plot for customer 55168

6.2 Force plot

This is the same information as the waterfall, laid out horizontally (Figure 1). We use the waterfall in reports and the force plot in dashboards, where space is wide rather than tall.

6.3 Mean SHAP plot

The mean |SHAP| bar plot ranks features by their average impact over all customers. Unlike the impurity-based importance in Part 2, it is measured in probability points, and it is consistent: a feature that matters more always gets a higher value. Next to it we show the bar plot of the selected customer's own SHAP values, so their profile can be compared with the global picture.

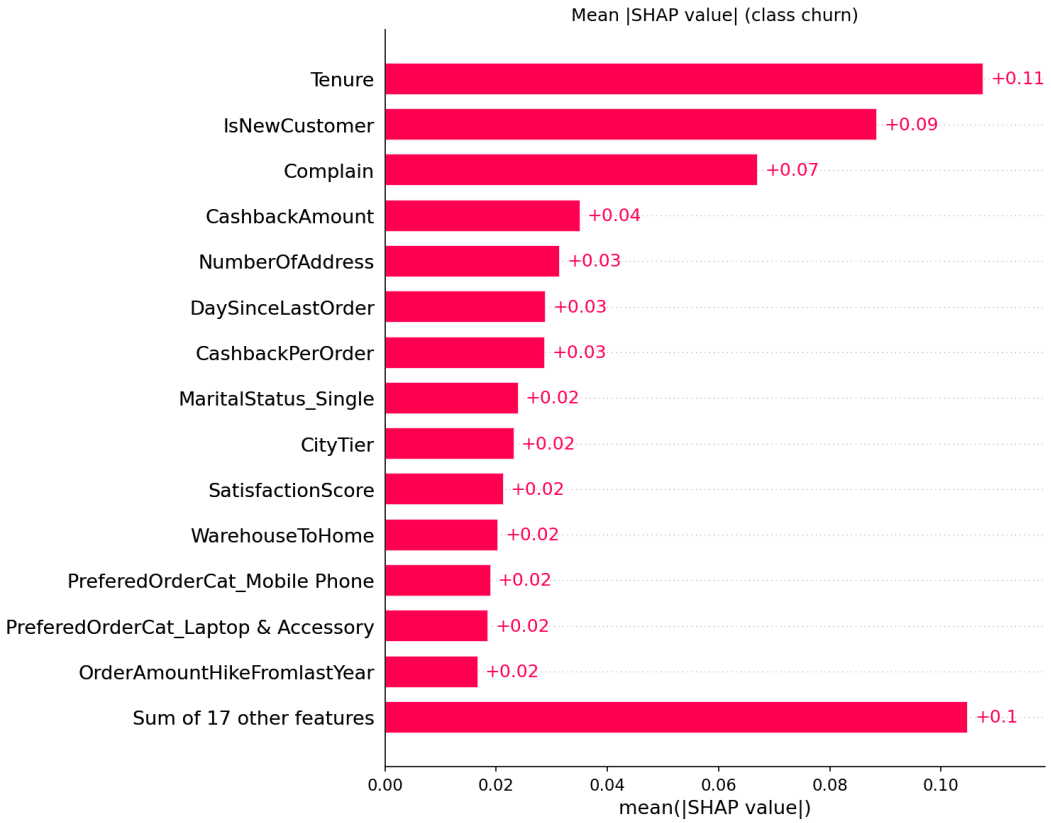


Figure 7 – Mean |SHAP value| (churn class), all test customers

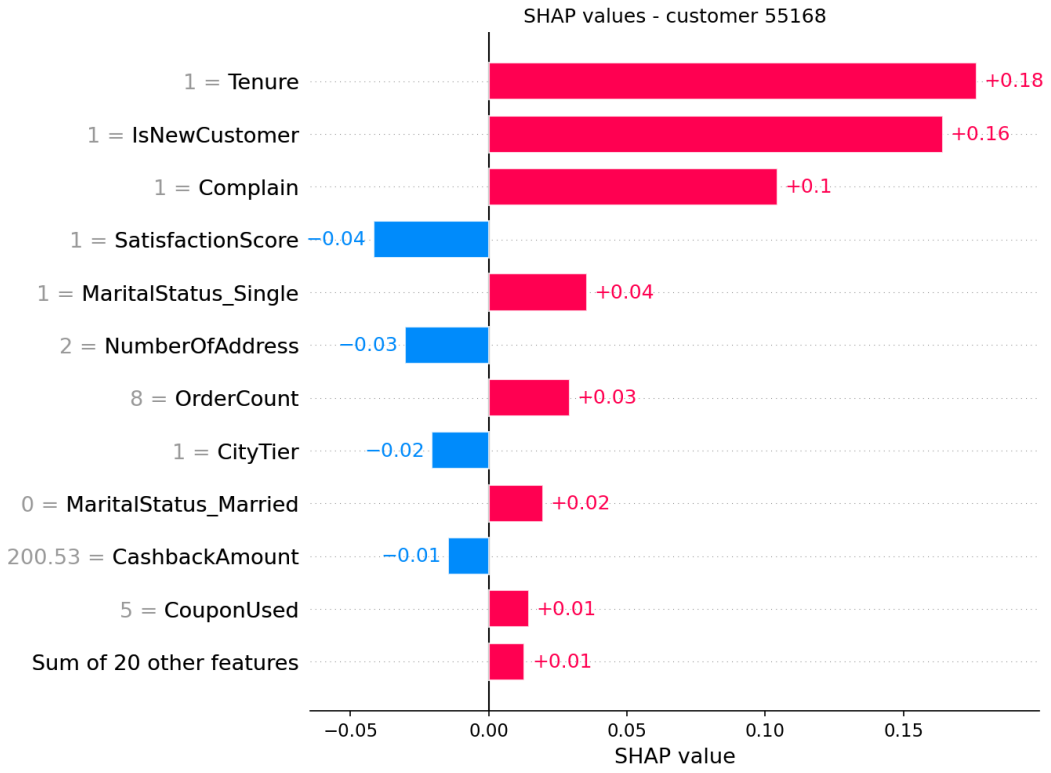


Figure 8 – SHAP values of customer 55168

6.4 Beeswarm plot

The beeswarm combines importance and direction in one picture: one dot per customer, stacked where values pile up.

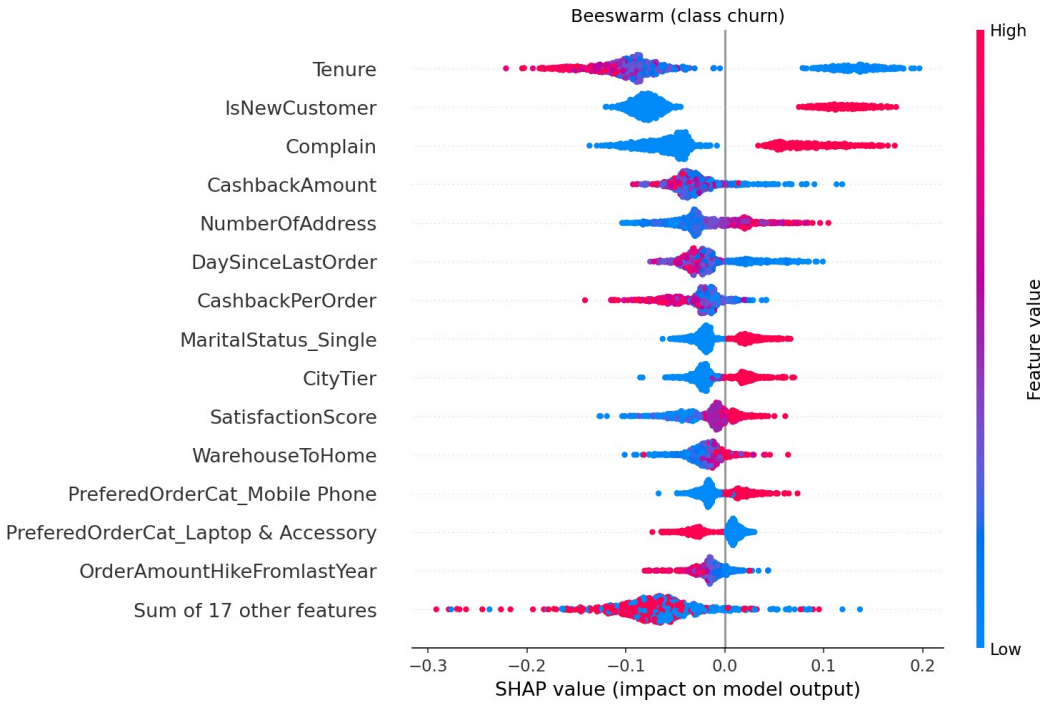


Figure 9 – Beeswarm plot (churn class)

6.5 Dependence plots

A dependence plot puts one feature on the x axis and its SHAP value on the y axis, for every customer. It shows the shape of the effect (straight line, threshold, plateau), and the colour shows the feature that interacts most with it. We plotted the four most important features and circled the selected customer in red to see where they sit.

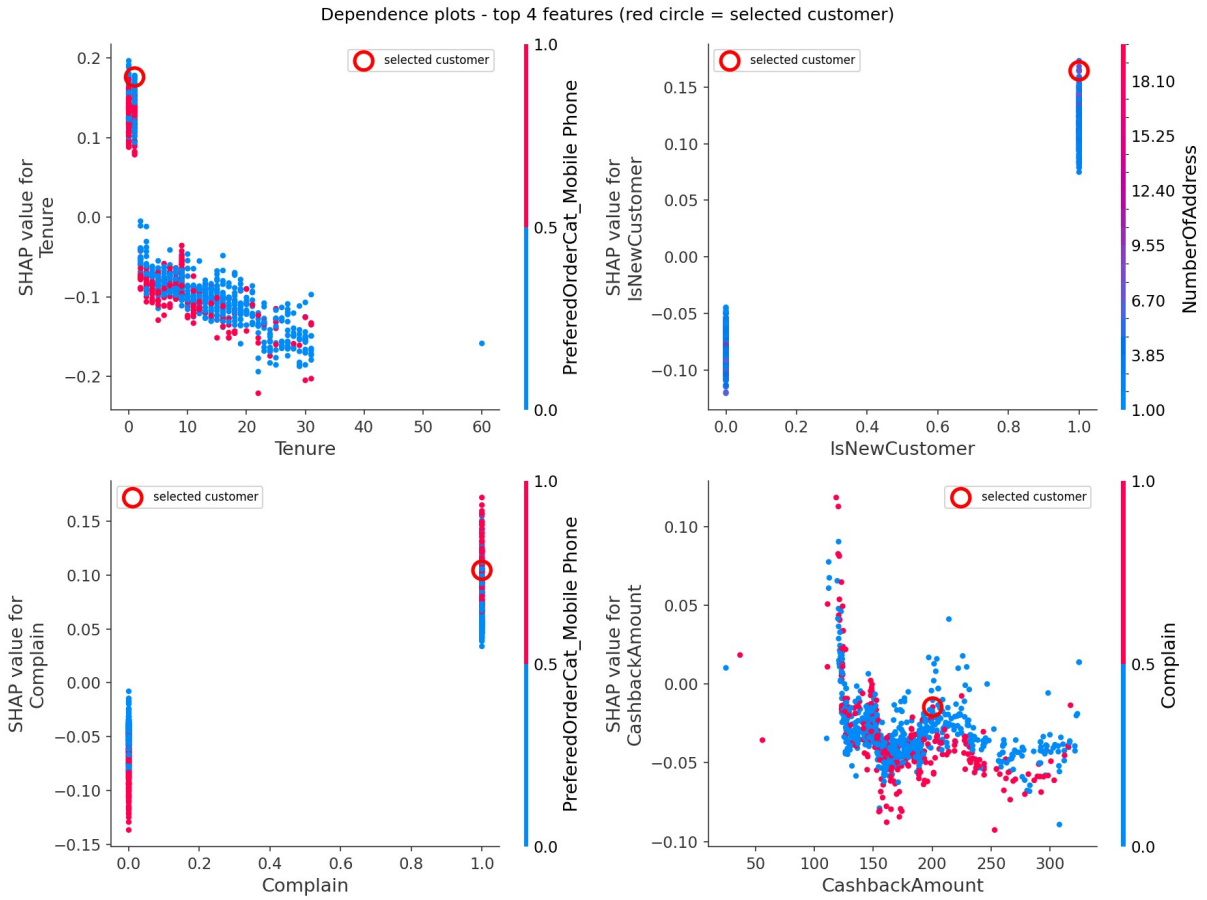


Figure 10 – Dependence plots of the top 4 features

7. What the explanations tell us

The table below is computed from the SHAP values. Direction is the correlation between a feature's value and its SHAP value across customers.

Feature	Mean |SHAP|	Direction (corr.)	Reading
Tenure	0.108	-0.73	higher value → less churn risk
IsNewCustomer	0.089	+0.98	being in this group raises churn risk
Complain	0.067	+0.94	being in this group raises churn risk
CashbackAmount	0.035	-0.36	higher value → less churn risk
NumberOfAddress	0.031	+0.84	higher value → more churn risk
DaySinceLastOrder	0.029	-0.56	higher value → less churn risk
CashbackPerOrder	0.029	-0.70	higher value → less churn risk
MaritalStatus_Single	0.024	+0.92	being in this group raises churn risk
CityTier	0.023	+0.91	higher value → more churn risk

SatisfactionScore	0.021	+0.83	higher value → more churn risk

The four features that matter most are Tenure, IsNewCustomer, Complain, CashbackAmount. Our exploration in Part 2 pointed to the same things (tenure, complaints and cashback), which gives us some confidence the model learned real patterns rather than noise. SHAP adds the direction and size of each effect, and shows how they differ from one customer to the next.

Things we noticed that matter for the business:

- **New customers are the main risk:** the first months are where customers are lost, so onboarding is the place to act.
- **Complaints are a strong signal:** a customer who complained is much more likely to leave, so a follow-up call after a complaint is a cheap retention action.
- **Some effects need a second look:** the satisfaction score effect should be checked with the business, as we noted in Part 2.

8. How RetailGenius would use this

Audience	What they get from SHAP
Marketing	Top 3 reasons per at-risk customer in the CRM, used to pick the right offer
Customer support	Plain-language reasons in the agent console ("complained last month")
Product / management	Mean SHAP and beeswarm plots in the monthly churn review
Data science	Dependence plots to debug the model and catch leakage
Legal / DPO	An explanation for each automated decision (GDPR transparency)

Limits. SHAP explains the model, not the real world: a feature with a high SHAP value is predictive, but that does not prove it causes churn. When features are correlated (for example Tenure and our IsNewCustomer flag), the credit is split between them, so their individual values should be read together. And because Part 2 found duplicated customers, we only explain the model trained on the de-duplicated data.

References

1. Lundberg, S. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS. https://arxiv.org/abs/1705.07874
2. Lundberg, S. et al. (2020). From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence 2, 56–67.
3. SHAP documentation. https://shap.readthedocs.io/
4. Molnar, C. (2022). Interpretable Machine Learning, chapter on SHAP. https://christophm.github.io/interpretable-ml-book/
5. Generative AI (Claude, Anthropic) was used to help write the code and parts of this report. The group reviewed, ran and edited everything.